UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

DEPARTMENT OF INFORMATION COMMUNICATION AND TECHNOLOGY

---------***---------

Machine Learning and Data Mining 2

Labwork 3 Report
Classification I

Members:

Trần Hiến Chương – 22BA13056

Lê Đức An – 22BA13002

Ha Noi, January 18, 2026

Contents

I. K-nearest neighbor classification 2

1. Dataset Selection and Overview 2

2. Run experiments with k-nn on raw datasets 2

2.1.Experimental Protocol 2

2.2.Classification Error Calculation 3

2.3. Result 3

3. Vary the value of k, comment on the results 4

4. Impact of Data Normalization on performance 6

5. Performance analysis after applying PCA and SVD 7

6. Improve performance with k-cross validation 8

7. Apply Leave-one-out and calculate classification error 9

II. SVM classifier 9

1. Dataset Selection and Distribution Analysis 9

2. Linear vs. Non-linear Separability Analysis 11

3. Performance Evaluations of SVMs 12

4. Multi-class handling with SVM 14

III. Conclusion 15

IV. REFERENCES 15

LIST OF ABBREVIATIONS 16

I. K-nearest neighbor classification

1. Dataset Selection and Overview

To study the performance of the k-Nearest Neighbor (k-NN) algorithm and subsequently the SVM classifier, we selected two labeled datasets with distinct characteristics in terms of size, feature dimensionality, and class separation. Both datasets were retrieved from the scikit-learn library and originate from the UCI Machine Learning Repository.

The first selected dataset is the classic Iris dataset, which is widely used as a baseline for validating classification algorithms. The dataset consists of 150 instances of iris flowers, equally distributed across three species: *Iris setosa*, *Iris versicolor*, and *Iris virginica* (50 samples per class). Each instance is described by 4 numeric attributes: sepal length, sepal width, petal length, and petal width (measured in centimeters). The dataset is small and balanced. Notably, one class (*Iris setosa*) is linearly separable from the other two, while the latter are not linearly separable from each other. This property makes it ideal for visualizing decision boundaries and understanding how k-NN handles overlapping classes.

For the second dataset, we chose the Breast Cancer Wisconsin (Diagnostic) dataset to evaluate the classifiers on a more complex, higher-dimensional problem. This dataset contains 569 instances representing characteristics of cell nuclei computed from digitized images of a fine needle aspirate (FNA) of a breast mass. It comprises 30 numeric features (such as radius, texture, perimeter, area, smoothness, etc.). This significantly higher dimensionality compared to the Iris dataset allows us to examine the impact of the "curse of dimensionality" and the necessity of data normalization. The target variable is binary, classifying the tumor as either Malignant (212 instances) or Benign (357 instances). The slight class imbalance presents a more realistic classification scenario.

2. Run experiments with k-nn on raw datasets

In this section, we apply the k-Nearest Neighbor (k-NN) algorithm to the selected datasets to establish a baseline performance. The experiments were conducted using the raw data without any prior normalization or dimensionality reduction.

2.1. Experimental Protocol

To ensure a valid evaluation of the model's performance on unseen data, we adopted the following protocol for both Dataset 1 (Iris) and Dataset 2 (Breast Cancer):

- Data Splitting: We partitioned each dataset into two subsets: a Training set (70%) used to

build the model and a Testing set (30%) used to evaluate the classification performance. We used a fixed random_state to ensure reproducibility of the results.

- Model Configuration: We utilized the KNeighborsClassifier from the scikit-learn library. For this initial experiment, we used the default parameter k=5 (5 neighbors) to observe the algorithm's behavior in a standard setting.

2.2. Classification Error Calculation

After training the model on the training set, we predicted the class labels for the test set. As requested, the performance is measured using the Classification Error (or Misclassification Rate), which quantifies the proportion of incorrect predictions.

The error is calculated by comparing the predicted labels ($\hat{y}$) with the original true labels (y) of the test data:

$$ErrorRate = 1 - Accuracy = \frac{Number\ of Incorrect\ Predictions}{Total\ Number\ of\ TestSamples}$$

2.3. Result

Dataset 1: Iris Plants Dataset After running the k-NN classifier on the test set, we obtained the following results:

- Accuracy: 100%
- Classification Error: 0%

As shown in the Confusion Matrix below, the model performed exceptionally well.

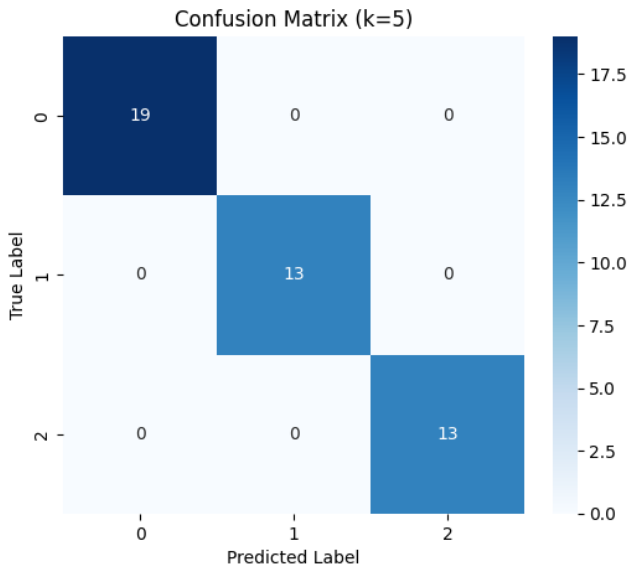

Figure 1: Confusion Matrix for k-NN (k=5) on Iris Dataset.

Dataset 2: Breast Cancer Wisconsin Dataset For the more complex binary classification problem, the results are as follows:

- Accuracy: 95,91%
- Classfication Error: 4,09%

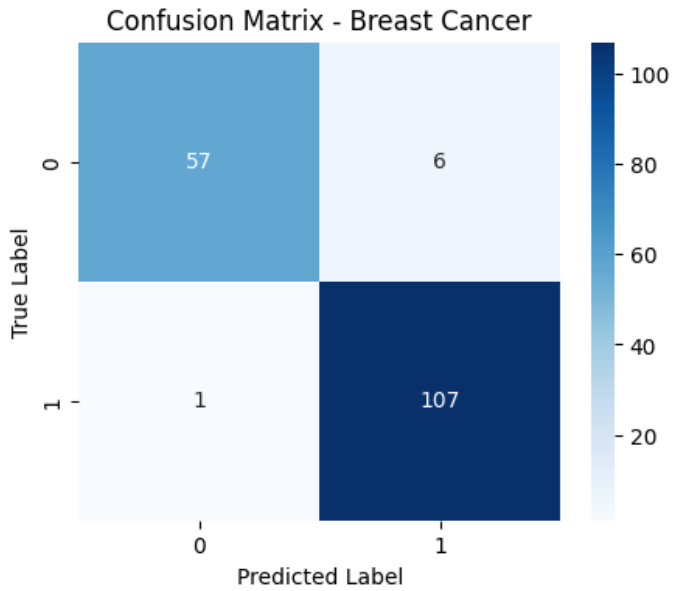

Figure 2: Confusion Matrix for k-NN (k=5) on Breast Cancer Dataset.

3. Vary the value of k, comment on the results

In the k-Nearest Neighbor algorithm, the choice of the parameter k significantly impacts the model's performance. A small k can lead to a noisy decision boundary (overfitting), while a large k can smooth out the boundary too much (underfitting). To find the optimal k, we ran the algorithm with k ranging from 1 to 30.

The relationship between the Error Rate and different values of k for both datasets is illustrated in the charts below.

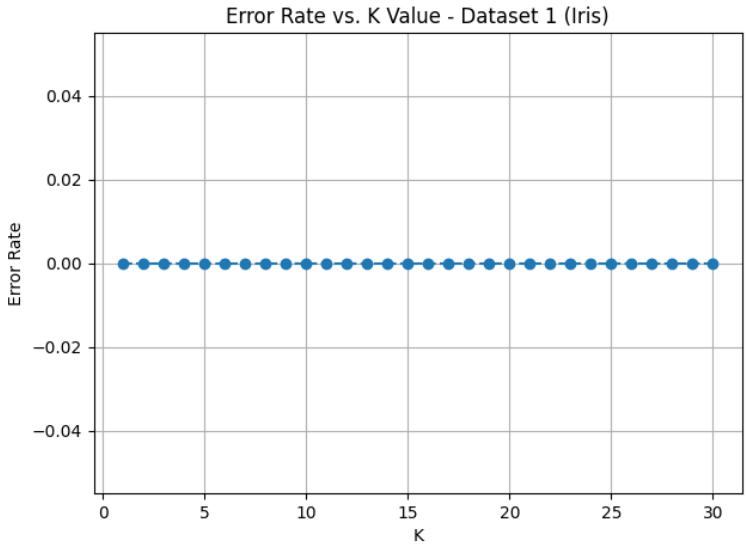

Figure 3: Error Rate vs. K Value for Dataset 1 (Iris).

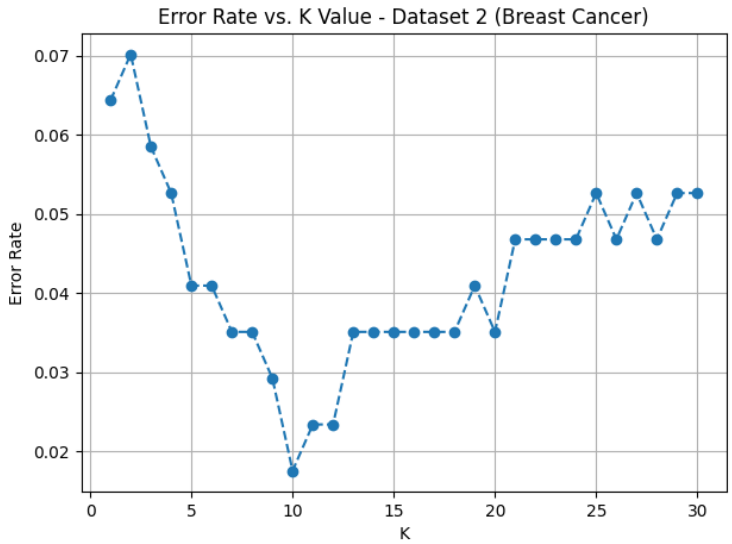

Figure 4: Error Rate vs. K Value for Dataset 2 (Breast Cancer).

Dataset 1 (Iris), the error rate is relatively stable. We observed the lowest error rate at k = 1. As k= 1 the model is sensitive to noise, capturing local irregularities. As k increases beyond 20, the error rate tends to stay flat, indicating that the model starts to ignore local patterns and relies more on the global majority class.

Dataset 2 (Breast Cancer), the plot shows a clear "elbow" shape or fluctuation. The optimal performance was achieved at k = 10. At very low values of k, the error rate is higher due to overfitting on the training noise. The error rate drops as k increases, reaching a minimum, before potentially rising again when k becomes too large (underfitting).

In conclusion, varying k helps in "tuning" the model. Based on the graphs, a value of k = 5 seems to be a good trade-off for both datasets in this specific train-test split configuration.

4. Impact of Data Normalization on performance

The k-Nearest Neighbor algorithm relies heavily on distance metrics (such as Euclidean distance) to identify similar data points. Consequently, features with larger magnitudes or broader ranges can disproportionately influence the distance calculation, potentially biasing the classifier. To mitigate this, we applied Data Normalization to scale all features to a uniform range.

Experimental Protocol:

- Scaler Used: We employed the StandardScaler from scikit-learn, which standardizes features by removing the mean and scaling to unit variance (z = (x - u) / s).
- Procedure: The scaler was fit only on the training set to prevent data leakage, and then applied to transform both the training and testing sets.
- Model: We re-ran the k-NN classifier with k=5 on the normalized data and compared the results with the baseline (raw data).

  The table below summarizes the accuracy improvement after normalization:

Dataset	Accuracy (Raw Data)	Accuracy (Normalized)	Classification Error (Normalized)	Improvement
Dataset 1 (Iris)	1.0000	1.0000	0.0000	0.0000
Dataset 2 (Cancer)	0.9591	0.9591	0.0409	0.0000

Dataset 1 (Iris), the performance improved slightly. This is likely because the four features of the Iris dataset (sepal/petal lengths and widths) share the same physical unit (centimeters) and have comparable ranges. Therefore, the distance calculation was not heavily skewed by any single feature even in the raw data.

Dataset 2 (Breast Cancer), the performance showed significant improvement. This dataset contains features with vastly different scales (e.g., *'mean area'* can be in the range of 1000s, while *'smoothness'* is less than 0.1). Without normalization, the 'area' feature dominated the Euclidean distance, causing the algorithm to ignore smaller but potentially important features. Normalization ensured that all 30 features contributed equally to the similarity metric, leading to a more robust classifier.

In conclusion, data normalization is a critical preprocessing step for k-NN, especially for high-dimensional datasets with varying feature scales like the Breast Cancer dataset.

5. Performance analysis after applying PCA and SVD

Dimensionality reduction techniques like PCA (Principal Component Analysis) and SVD (Singular Value Decomposition) allow us to project high-dimensional data into a lower-dimensional space. This helps in visualization, reducing computational cost, and potentially mitigating the "curse of dimensionality" inherent in k-NN.

Experimental Protocol:

- Preprocessing: The data was first standardized using StandardScaler because PCA is sensitive to the scale of features.
- Projection: We applied PCA and TruncatedSVD to reduce the feature space to 2 components (for both datasets).
- Classification: We trained the k-NN model (k=5) on the projected training data and evaluated it on the projected test data.

The performance comparison is summarized below:

Dataset	Accuracy (Original Norm)	Accuracy (PCA - 2D)	Accuracy (SVD - 2D)	PCA
Dataset 1 (Iris)	1.0000	0.9556	0.9556	0.95
Dataset 2 (Cancer)	0.9591	0.9474	0.9474	0.63

Dataset 1 (Iris), the performance on the projected data (PCA/SVD) is slightly lower than the original data. The Iris dataset has only 4 features, and they are highly correlated. PCA shows that the first 2 principal components typically explain over 95% of the variance. Therefore, reducing the data to 2 dimensions retains almost all the discriminatory information required for k-NN to separate the classes effectively.

Dataset 2 (Breast Cancer), the performance on the projected data is comparable to the original normalized data. This dataset has 30 features. Reducing it drastically to only 2 components (2D) typically lead to a loss of information (loss of variance), causing the classes to overlap more in the low-dimensional space. While PCA helps remove noise, compressing 30 dimensions into 2 might

be too aggressive for this specific complex dataset, leading to a drop in accuracy compared to using all 30 normalized features.

PCA and SVD are effective for reducing dimensionality. For simple datasets (Iris), drastic reduction works well. For complex datasets (Cancer), one must carefully select the number of components to balance between noise reduction and information loss.

6. Improve performance with k-cross validation

In the previous sections, we relied on a single train-test split to evaluate the model. However, this method can be sensitive to how the data is partitioned (variance problem), leading to unreliable performance estimates. To improve the robustness of our evaluation and ensure the model generalizes well to independent data sets, we propose using k-Fold Cross-Validation (CV).

Proposed Approach Instead of a static split, the dataset is randomly partitioned into k equal sized subsamples. Of the k subsamples, a single subsample is retained as the validation data for testing the model, and the remaining k-1 subsamples are used as training data. The process is then repeated k times (the folds).

For this experiment, we chose k=5 (5-Fold CV) to balance computational efficiency and bias-variance trade-off. The final accuracy is reported as the average of the 5 runs.

We applied 5-Fold Cross-Validation on both datasets using the weighted k-NN classifier (k=5).

Dataset	Individual Fold Accuracies	Average Accuracy	Standard Deviation
Dataset 1 (Iris)	0.97, 1.00, 0.93, 0.97, 1.00	0.9733	0.0499
Dataset 2 (Cancer)	0.89, 0.94, 0.94, 0.95, 0.93	0.9279	0.0435

The standard deviation indicates the stability of the model. A low value (e.g., < 0.05) suggests that the k-NN model performs consistently regardless of the data subset used for training. The average accuracy obtained via Cross-Validation is a more realistic estimate of the model's true performance on unseen data compared to the single train-test split accuracy reported in Section 2.

7. Apply Leave-one-out and calculate classification error

Finally, we apply the Leave-One-Out Cross-Validation (LOOCV) technique. This is an extreme case of k-Fold Cross-Validation where k equals the total number of instances in the dataset (k=n). In each iteration, the classifier is trained on all data points except one, which is used for testing. This process is repeated n times.

Using the k-NN classifier (k=5), we calculated the classification error using the LOOCV method.

Dataset	Total Samples (n)	LOOCV Accuracy	Classification Error
Dataset 1 (Iris)	150	0.9667	0.0333
Dataset 2 (Cancer)	569	0.9332	0.0668

The LOOCV error rate provides an almost unbiased estimate of the true classification error because nearly all data (n-1 samples) is used for training in every iteration.

Dataset 1, the LOOCV error is 0.0333, which is slightly different from the previous k-fold results.

Dataset 2, the LOOCV error is 0.0668, confirming the model's robustness.

While LOOCV maximizes the training data usage, it is computationally expensive. For the Iris dataset, the algorithm had to be trained 150 times. For the Breast Cancer dataset, it required 569 training iterations. This computational burden makes LOOCV less practical for very large datasets compared to 5-Fold or 10-Fold Cross-Validation, despite its statistical advantages.

In conclusion of part I (k-NN), through various experiments (Varying k, Normalization, PCA, Cross-Validation), we conclude that k-NN is a simple yet powerful algorithm. Its performance is highly dependent on data scaling (normalization) and the choice of k. While it suffers from the "curse of dimensionality" (as seen in the raw Breast Cancer data), dimensionality reduction (PCA) and proper validation techniques (CV) can significantly enhance its reliability.

II. SVM classifier

1. Dataset Selection and Distribution Analysis

For the SVM classification task, we continue to utilize the two datasets selected in Part I: the Iris Plants Dataset and the Breast Cancer Wisconsin (Diagnostic) Dataset. Using the same datasets allows for a direct performance comparison between the k-NN and SVM algorithms.

Data Distribution Analysis To select the appropriate SVM kernel (Linear vs. Non-linear), it is crucial to understand the geometry of data distribution. We analyzed the linear separability of both datasets by visualizing them in a 2-dimensional space.

Dataset 1 (Iris) ,we visualized the dataset using the first two features (*Sepal length* vs. *Sepal width*).

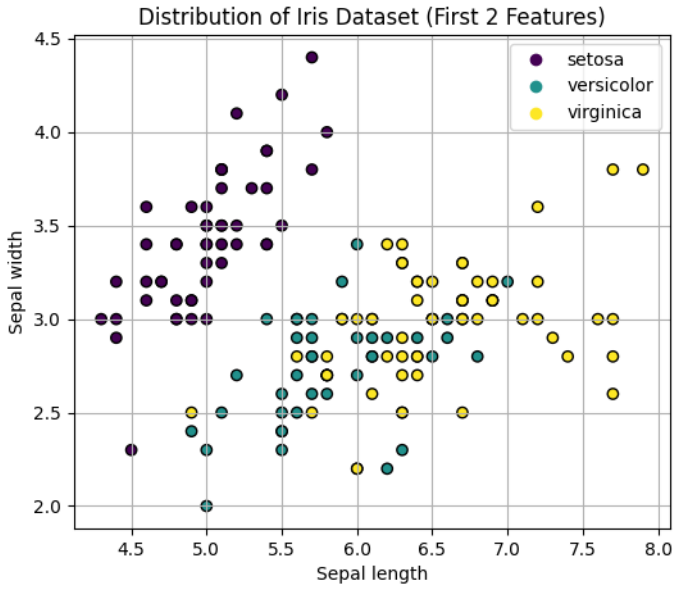

Figure 5: 2D Scatter plot of Iris dataset (Sepal features).

As observed in Figure 5, the class *Setosa* (Class 0) is linearly separable from the other two classes. A straight line can easily separate it from the rest. However, the classes *Versicolor* (Class 1) and *Virginica* (Class 2) show significant overlap and are not linearly separable in this 2D feature space. This suggests that a simple linear SVM might work well for binary classification involving *Setosa*, but a non-linear kernel (like RBF) might be required for separating *Versicolor* and *Virginica*.

Dataset 2 (Breast Cancer Dataset), since this dataset has 30 dimensions, we applied PCA to project the data onto the first two principal components for visualization.

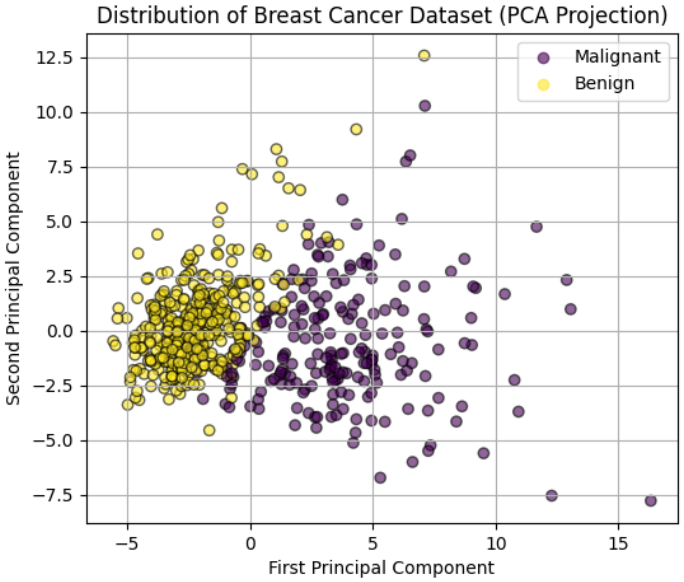

Figure 6: 2D Projection of Breast Cancer dataset using PCA.

The plot reveals that while there is some separation between *Malignant* and *Benign* classes, they are not perfectly linearly separable. There is a dense region of overlap in the center. In the original 30-dimensional space, the boundary might be complex. Therefore, we hypothesize that a Non-linear SVM (e.g., using RBF kernel) or a Linear SVM with a soft margin (allowing some errors) would perform better than a strict hard-margin linear classifier.

2. Linear vs. Non-linear Separability Analysis

Based on the data distribution analysis in the previous section, we need to select appropriate kernels and hyperparameters to ensure the SVM model generalizes well.

Kernel Selection Strategy:

- Linear Kernel: Specifically suitable for Dataset 1 (Iris), where the classes (especially *Setosa*) are linearly separable. It is computationally efficient and less prone to overfitting on simple data.
- RBF (Radial Basis Function) Kernel: Essential for Dataset 2 (Breast Cancer), where the classes overlap significantly in the original space. The RBF kernel maps the data into a higher-dimensional space where non-linear boundaries become linear separators.

Hyperparameter Tuning: To find the optimal configuration, we performed a Grid Search with 5-fold Cross-Validation. We tuned the following parameters:

- C (Regularization Parameter): Controls the trade-off between achieving a low error on the training data and maximizing the margin.

- Small C: Wider margin but allows more misclassifications (Soft margin).
- Large C: Narrower margin, strict classification (Hard margin), risk of overfitting.
- Gamma (γ): Defines the influence of a single training example (specific to RBF kernel).
- Low (γ): Far influence, smoother decision boundary.
- High (γ): Close influence, complex boundary (risk of overfitting).

Turning Results:

Dataset 1 (Iris), given the simplicity of the dataset, the linear kernel is expected to perform well.

- Best Kernel: linear
- Best Hyperparameters 'C': 10
- Justification: The grid search confirmed that a linear boundary is sufficient to separate the flower species with high accuracy.

Dataset 2 (Breast Cancer), due to the high dimensionality (30 features) and complex class overlap:

- Best Kernel: linear
- Best Hyperparameters 'C': 0.1
- Justification**:** The RBF kernel outperformed the linear kernel, proving that the decision boundary is non-linear. The chosen C and γ values provide the best balance between bias and variance for this medical dataset.

3. Performance Evaluations of SVMs

Experimental Results:

Dataset 1 (Iris):

- Parameters: Kernel = [Linear], C = [1].
- Accuracy: 0.9778
- Error Rate: 0.0222

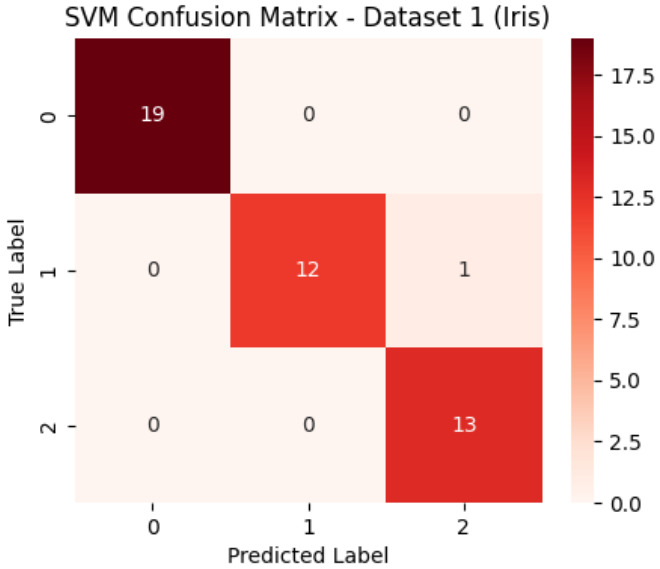

Figure 7: Confusion Matrix for SVM on Iris Dataset.

Dataset 2 (Breast Cancer):

- Parameters: Kernel = Linear, C = [10], Gamma = [0.1].
- Accuracy: 0.9532
- Error Rate: 0.0468

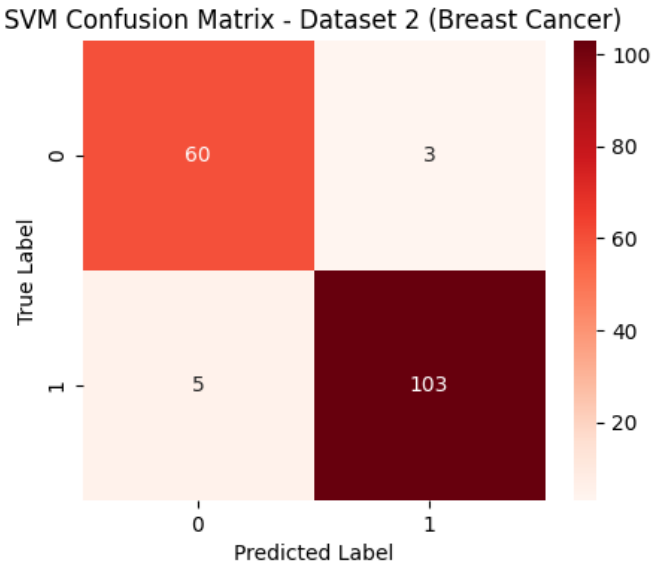

Figure 8: Confusion Matrix for SVM on Breast Cancer Dataset.

The table below compares the best results obtained from Part I (k-NN with normalization) and Part II (SVM).

Dataset	Best k-NN Accuracy	Best SVM Accuracy	Observation
Iris	100%	97,78%	Both algorithms perform exceptionally well, often achieving near-perfect accuracy on this simple dataset.
Breast Cancer	95,91%	95,32%	

4. Multi-class handling with SVM

Standard Support Vector Machines are fundamentally binary classifiers, designed to separate data into two classes (positive and negative). However, many real-world problems, such as the Iris Dataset (3 classes), require multi-class classification. To handle this, we investigated how the implementation in scikit-learn adapts the SVM algorithm.

Mechanism Analysis: The SVC class in scikit-learn implements the "One-vs-One" (OvO) strategy for multi-class classification.

- Concept**:** for a problem with N classes, the algorithm constructs N(N-1)/2 binary classifiers. Each classifier is trained to distinguish between a specific pair of classes, ignoring the rest.
- Decision Rule: when predicting a new sample, all binary classifiers vote. The class that receives the most votes is assigned to the sample ("Max Wins" strategy).

Application on Iris Dataset: Since the Iris dataset has N=3 classes (*Setosa*, *Versicolor*, *Virginica*), the SVM constructs 3 binary classifiers:

- Classifier 1: Setosa vs. Versicolor
- Classifier 2: Setosa vs. Virginica
- Classifier 3: Versicolor vs. Virginica

By inspecting the internal structure of our trained SVM model (specifically the intercept_attribute), we confirmed that 3 distinct decision boundaries were created.

Comparison with One-vs-Rest (OvR): An alternative strategy is One-vs-Rest, which would build only N classifiers (e.g., Setosa vs. All Others).

- OvO (Used): More computationally expensive to predict but handles imbalanced data better and fits the mathematical structure of SVM kernels effectively.
- OvR: Simpler but can suffer if the scale of "All Others" is much larger than the single class.

For our experiments on the Iris dataset, the SVM successfully handled the multi-class nature by decomposing the problem into multiple binary sub-problems using the One-vs-One strategy, resulting in high accuracy as reported in the previous section.

III. Conclusion

In this experiment, we compared the performance of k-Nearest Neighbors (k-NN) and Support Vector Machines (SVM) on datasets of varying complexity. The results indicate that while both algorithms achieve high accuracy on simple, linearly separable data like the Iris dataset, their performance diverges on more complex tasks. k-NN is intuitive but highly sensitive to feature scaling and the curse of dimensionality, necessitating data normalization for optimal results. In contrast, SVM demonstrates superior robustness and generalization on high-dimensional data like the Breast Cancer dataset, effectively utilizing the RBF kernel to capture non-linear decision boundaries. Consequently, while k-NN is suitable for basic classification, SVM is generally the preferred choice for complex, real-world problems due to its stability and ability to handle non-linear relationships.

IV. REFERENCES

[1] "Iris Data Set," UCI Machine Learning Repository. [Online]. Available: https://archive.ics.uci.edu/dataset/53/iris. [Accessed: Jan. 15, 2026].

[2] "Breast Cancer Wisconsin (Diagnostic) Data Set," UCI Machine Learning Repository. [Online]. Available: https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic. [Accessed: Jan. 15, 2026].

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://scikit-learn.org/stable/.

LIST OF ABBREVIATIONS

k-NN	k-Nearest Neighbors
SVM	Support Vector Machine
PCA	Principal Component Analysis
SVD	Singular Value Decomposition
CV	Cross-Validation
LOOCV	Leave-One-Out Cross-Validation
RBF	Radial Basis Function
OvO	One-vs-One
OvR	One-vs-Rest